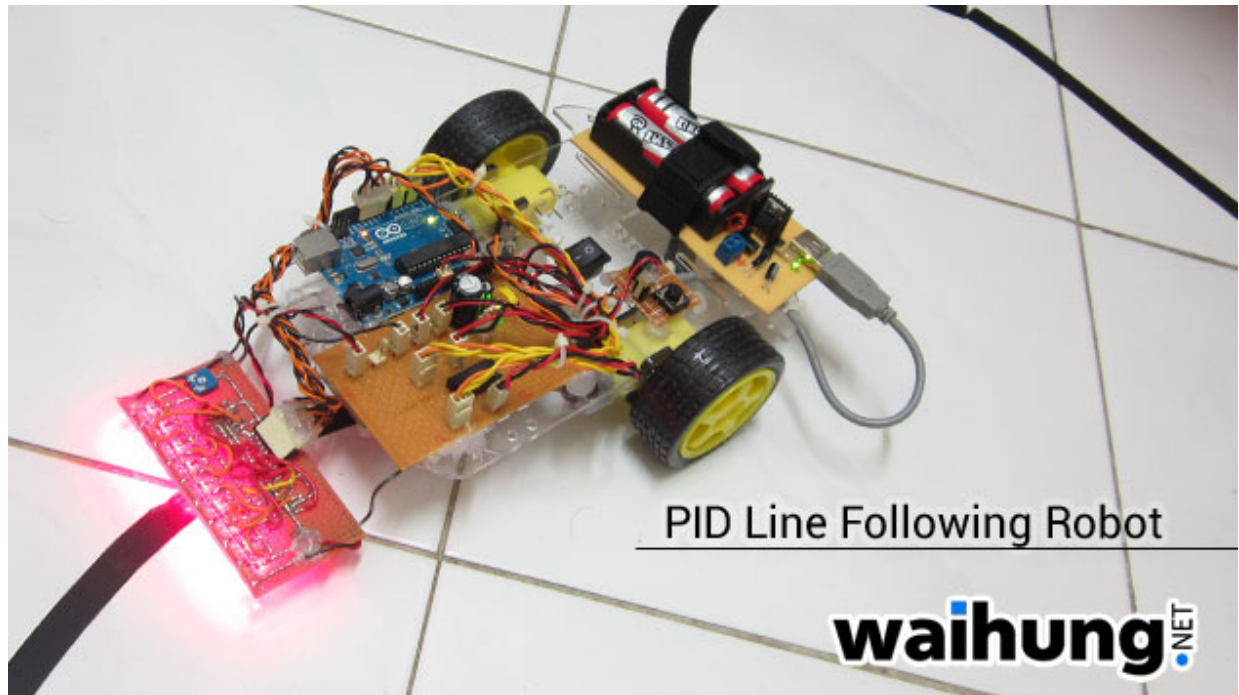


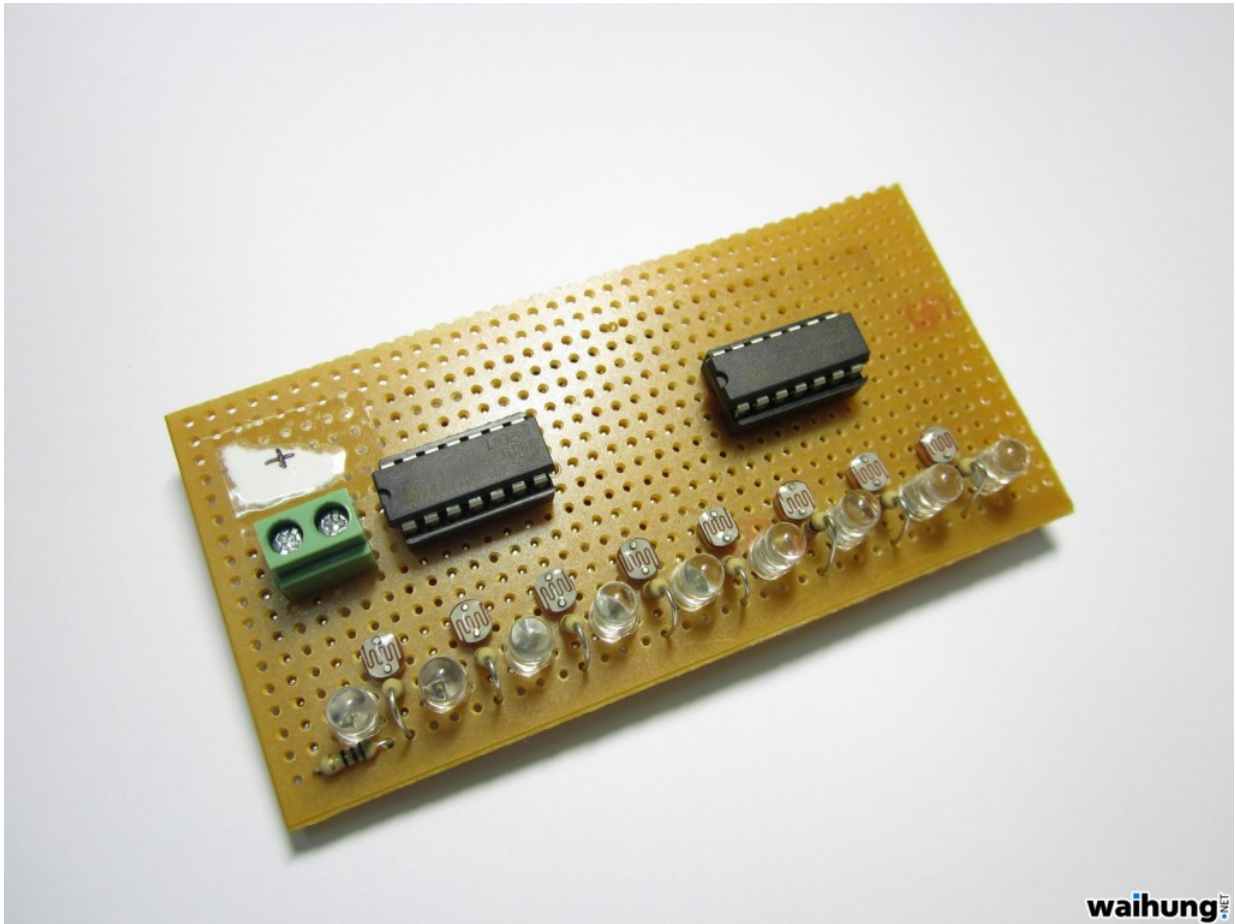
PID Line Following Robot

 waihung.net/pid-line-following-robot/



Previously, I've made a Bluetooth Controlled Robot ([read post](#)). By adding a line sensor to the robot, I've managed to create a simple line following robot and utilize the ever popular PID algorithm to navigate the robot.

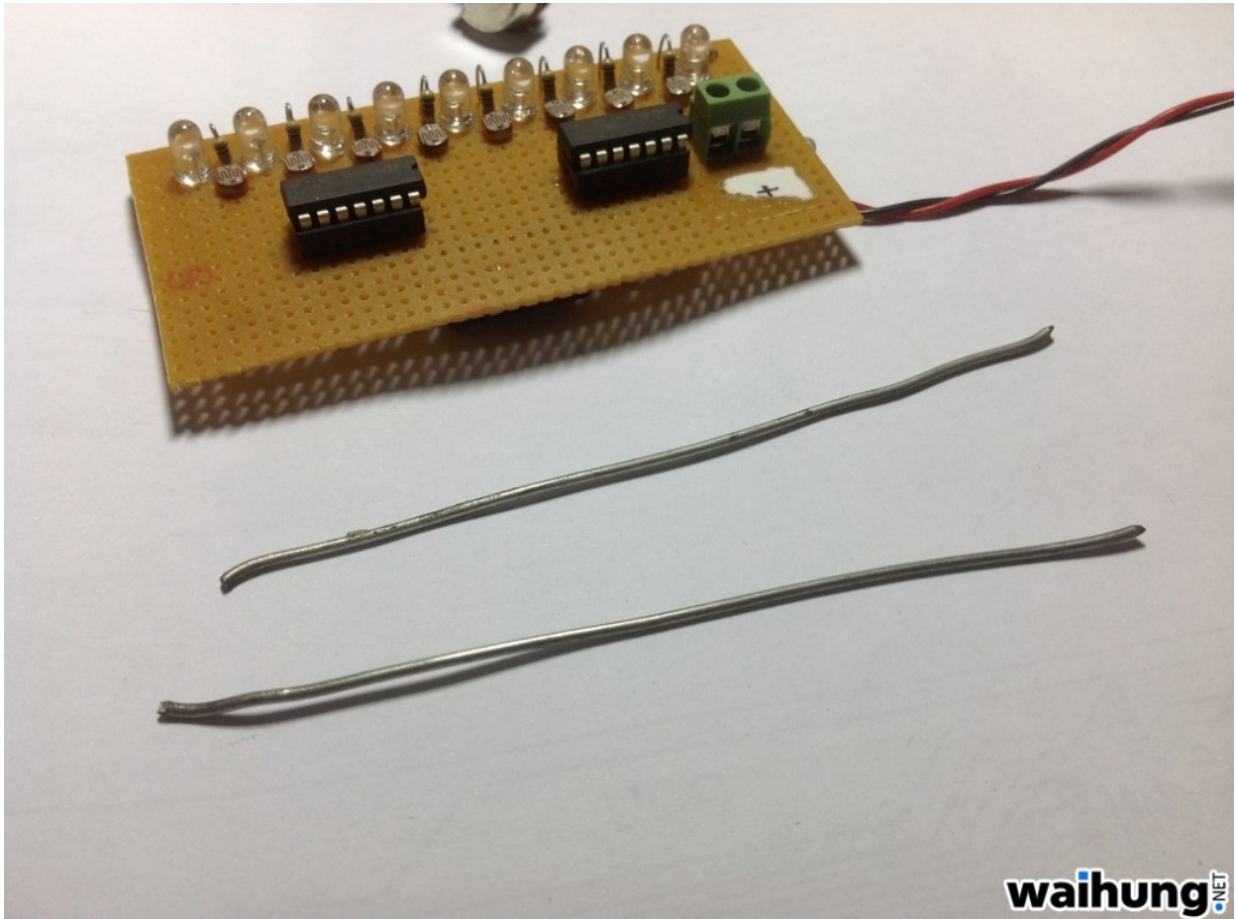
You can read more about my DIY line following sensor [here](#) and how I made it.



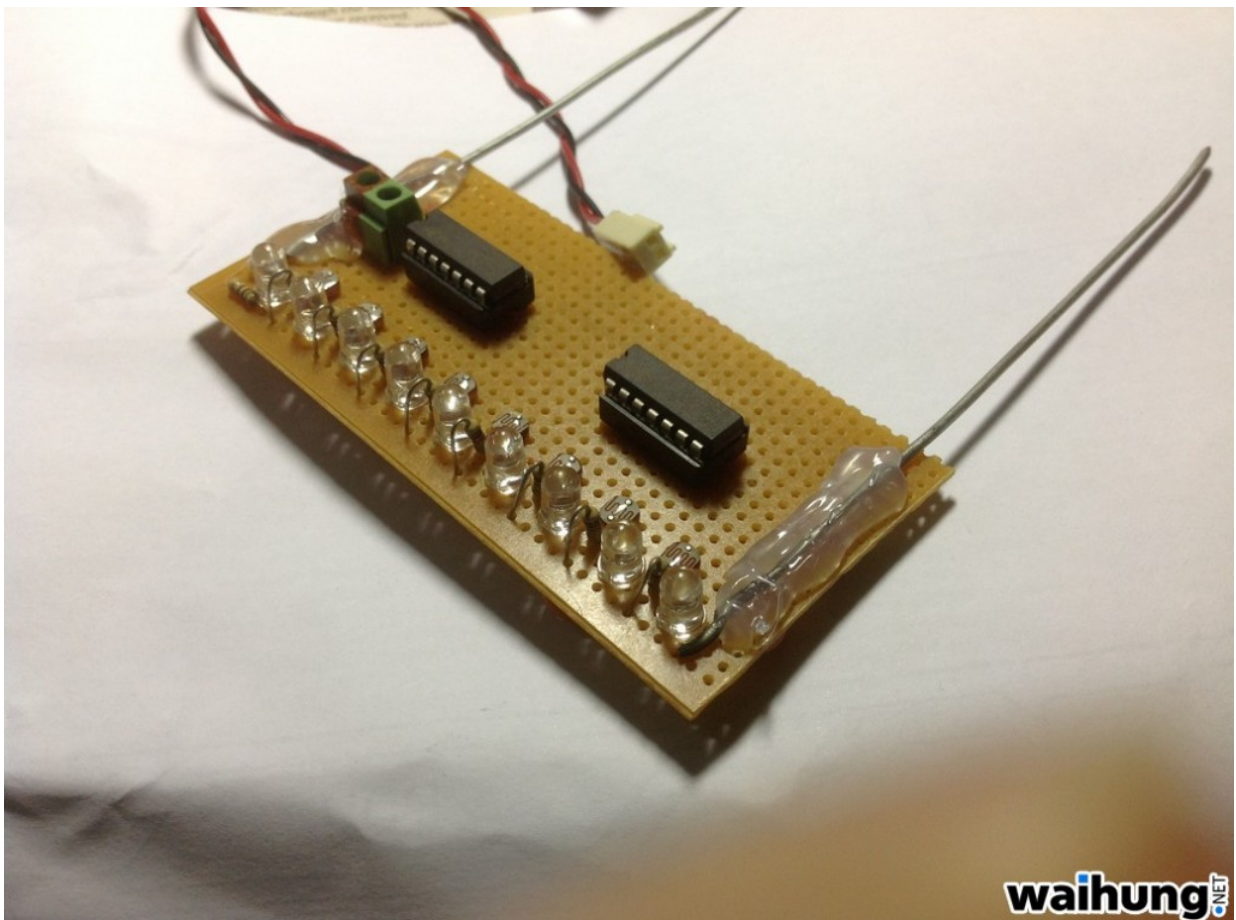
waihung.NET

Now it's time to mount this line following sensor unto the robot.

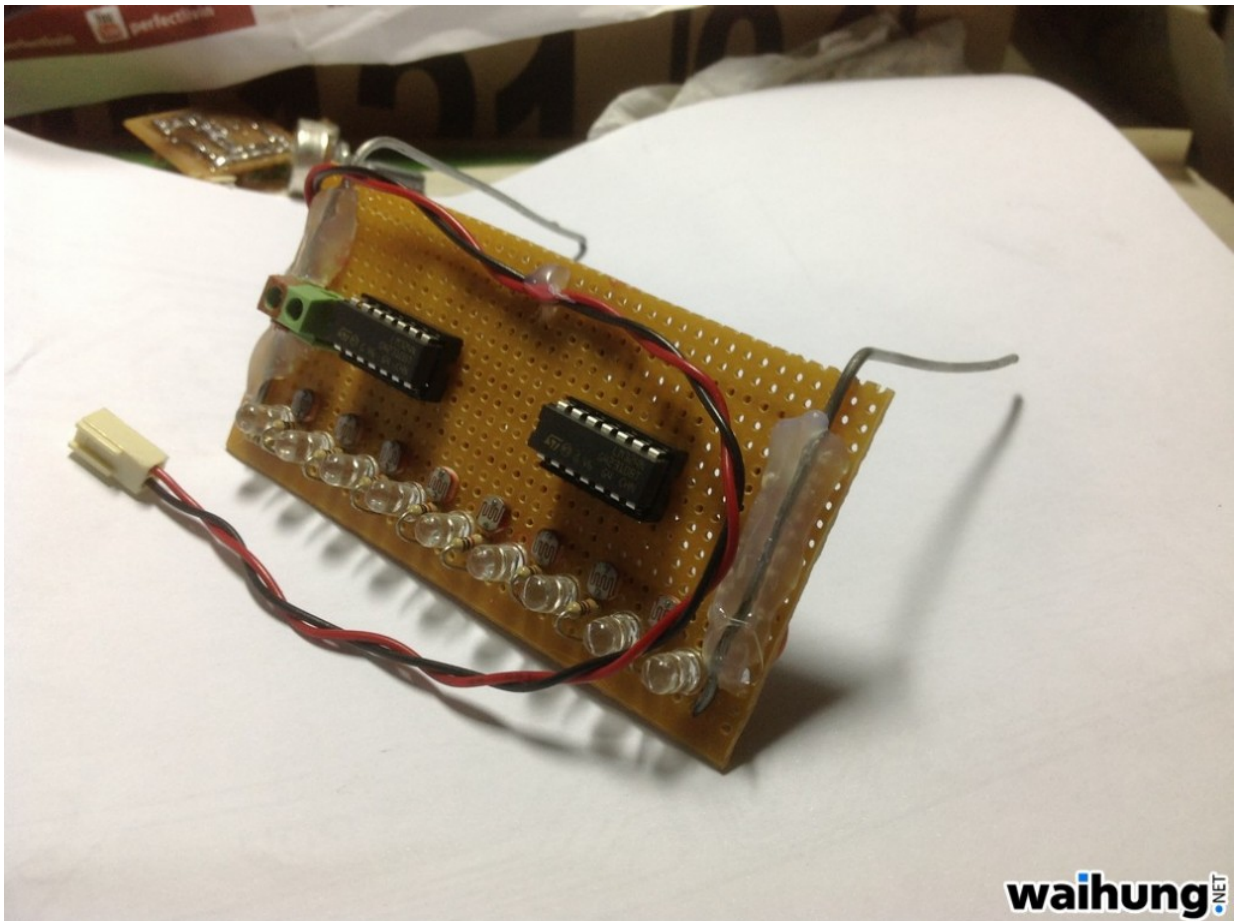
I got myself two pieces of relatively thick and rigid iron rod.



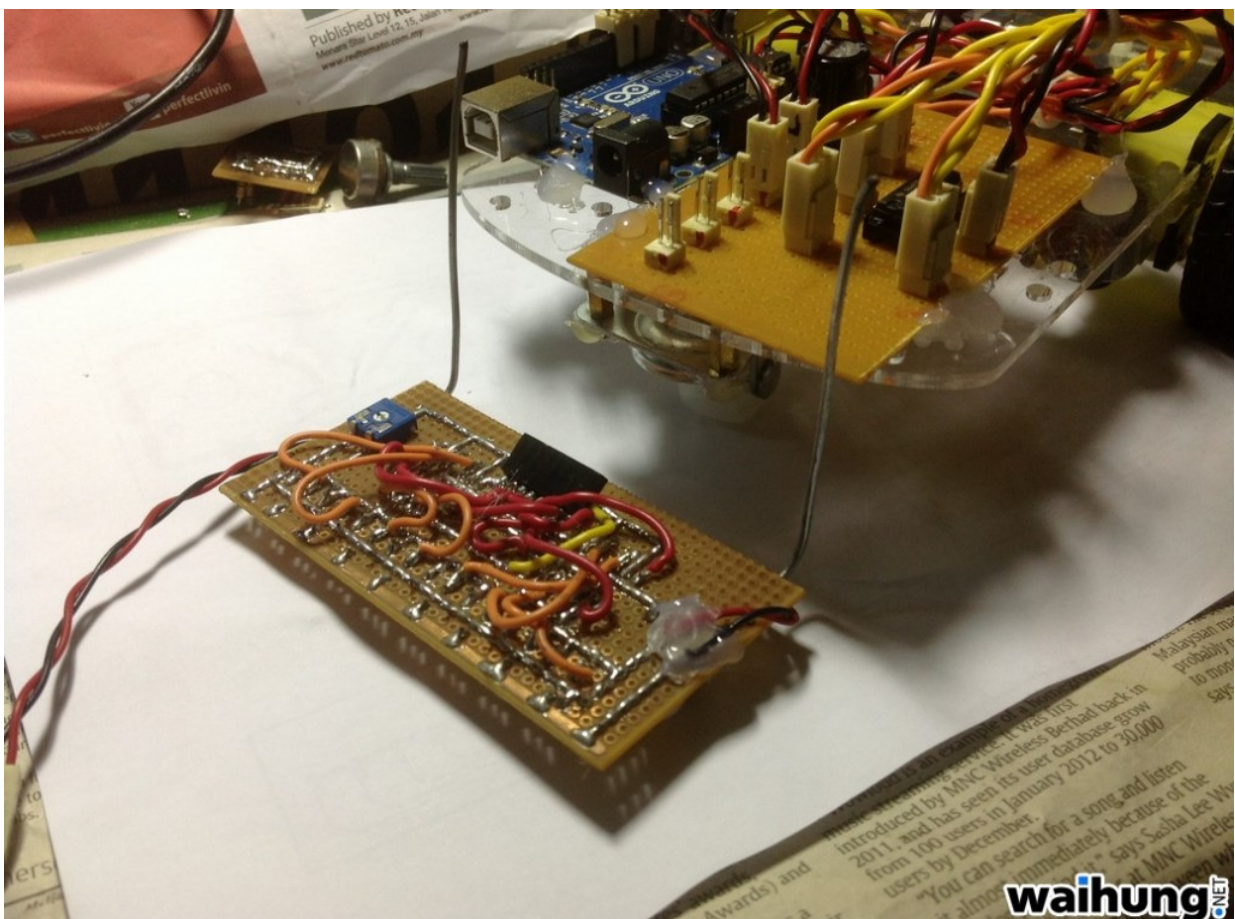
And hot glue them onto the line following sensor. (Hot glue FTW!)



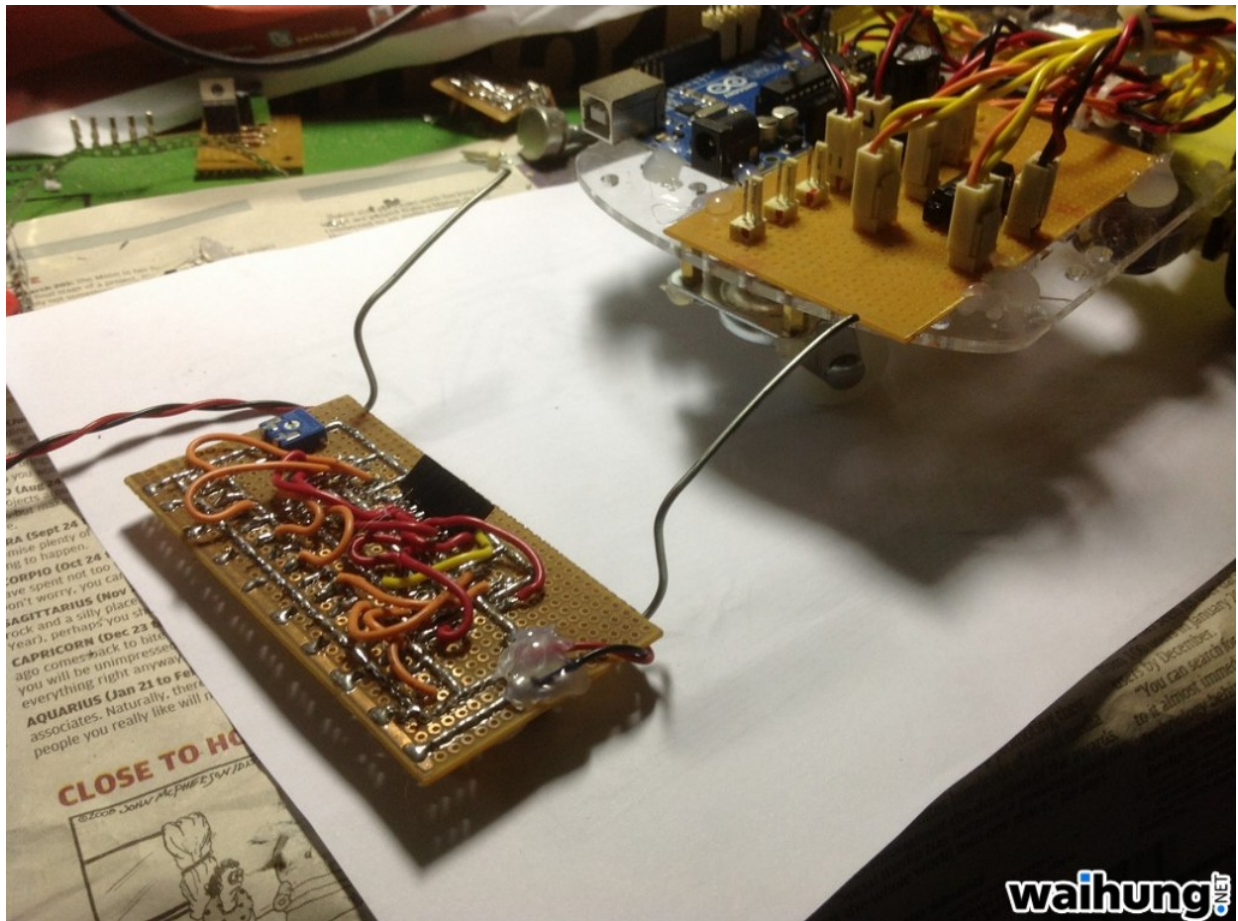
Bend the iron rods to 90 degree.



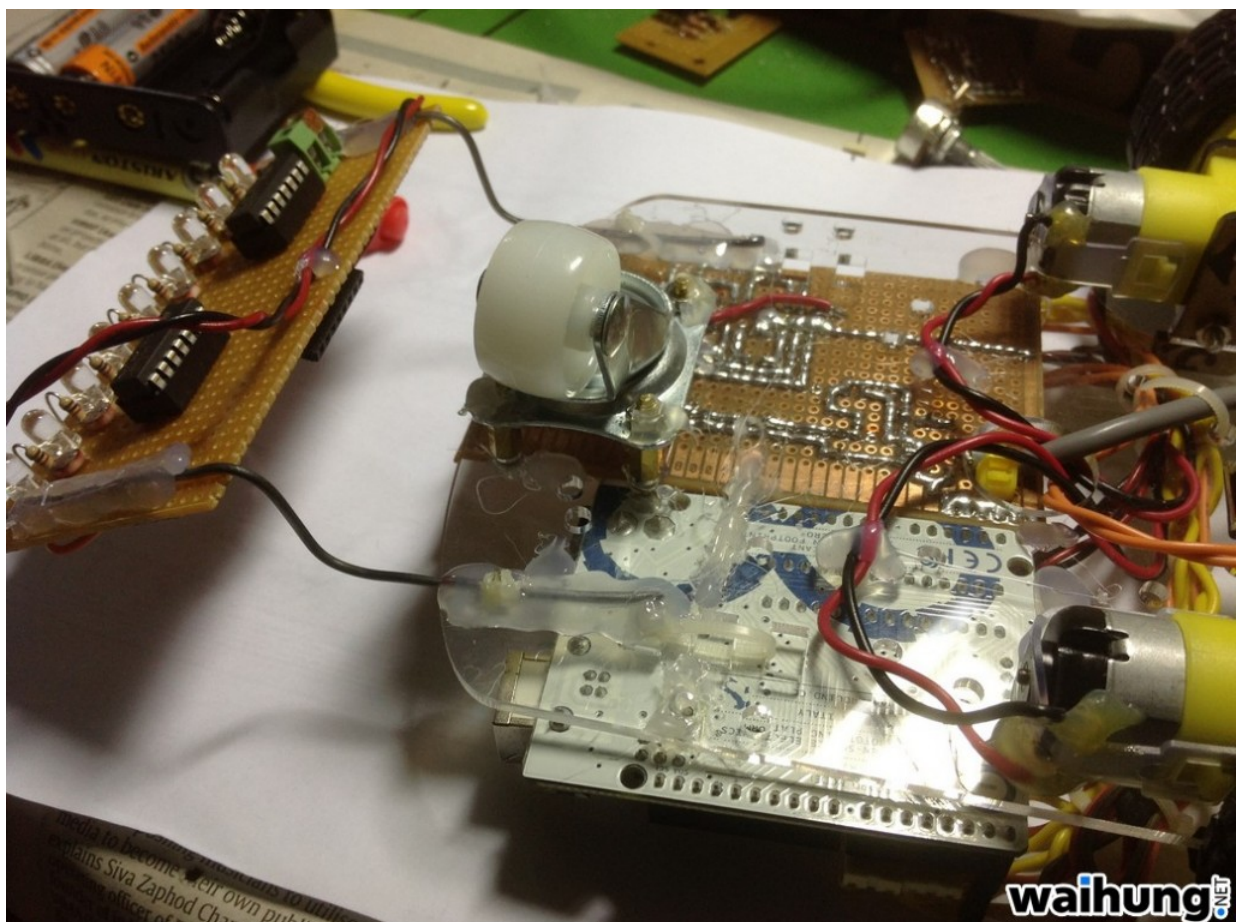
And flip it over to estimate how high will it be mounted on the robot.



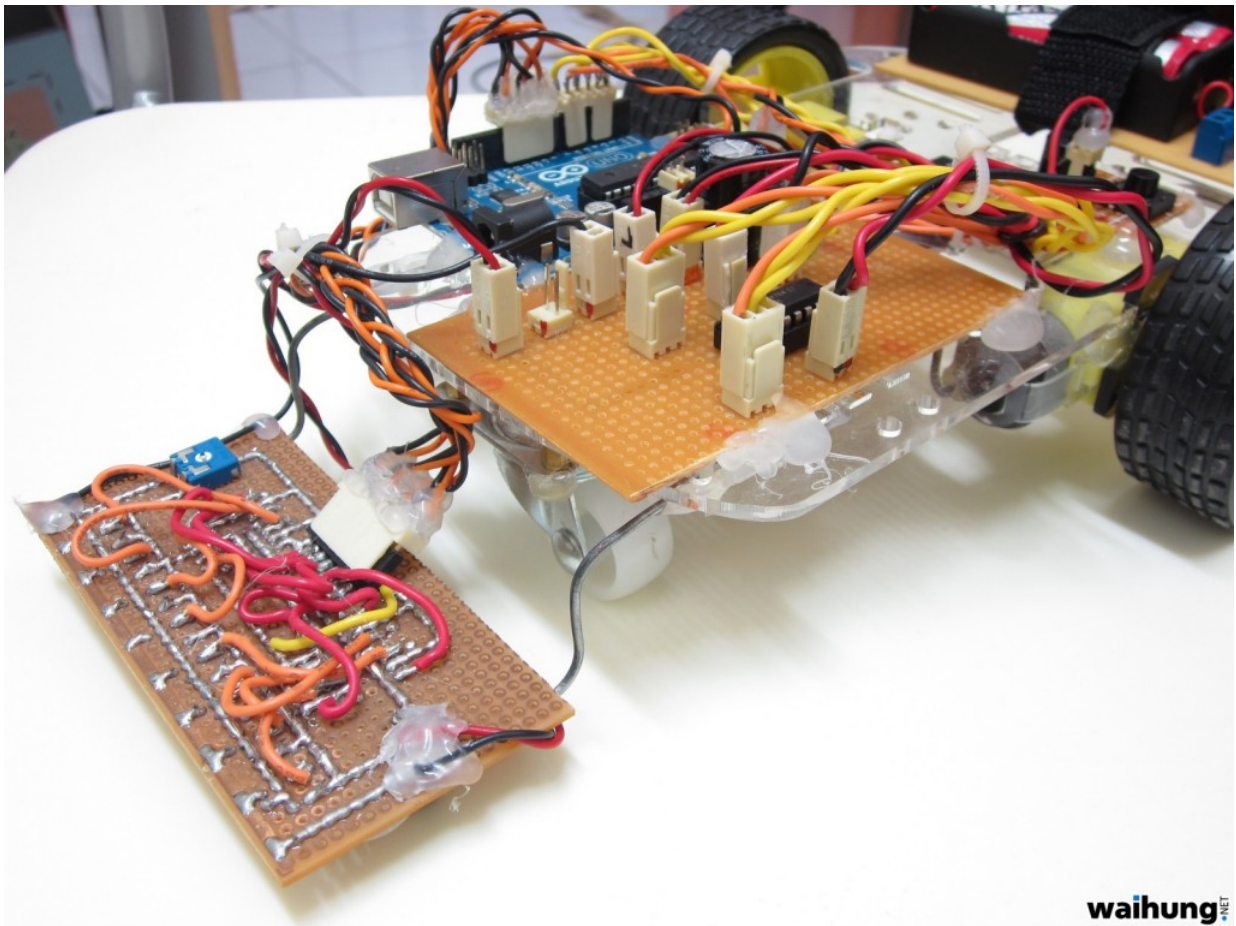
After making sure the height is correct, mark the point and make the second bend.



Next, use hot glue (again) to mount the iron rods to the robot base.

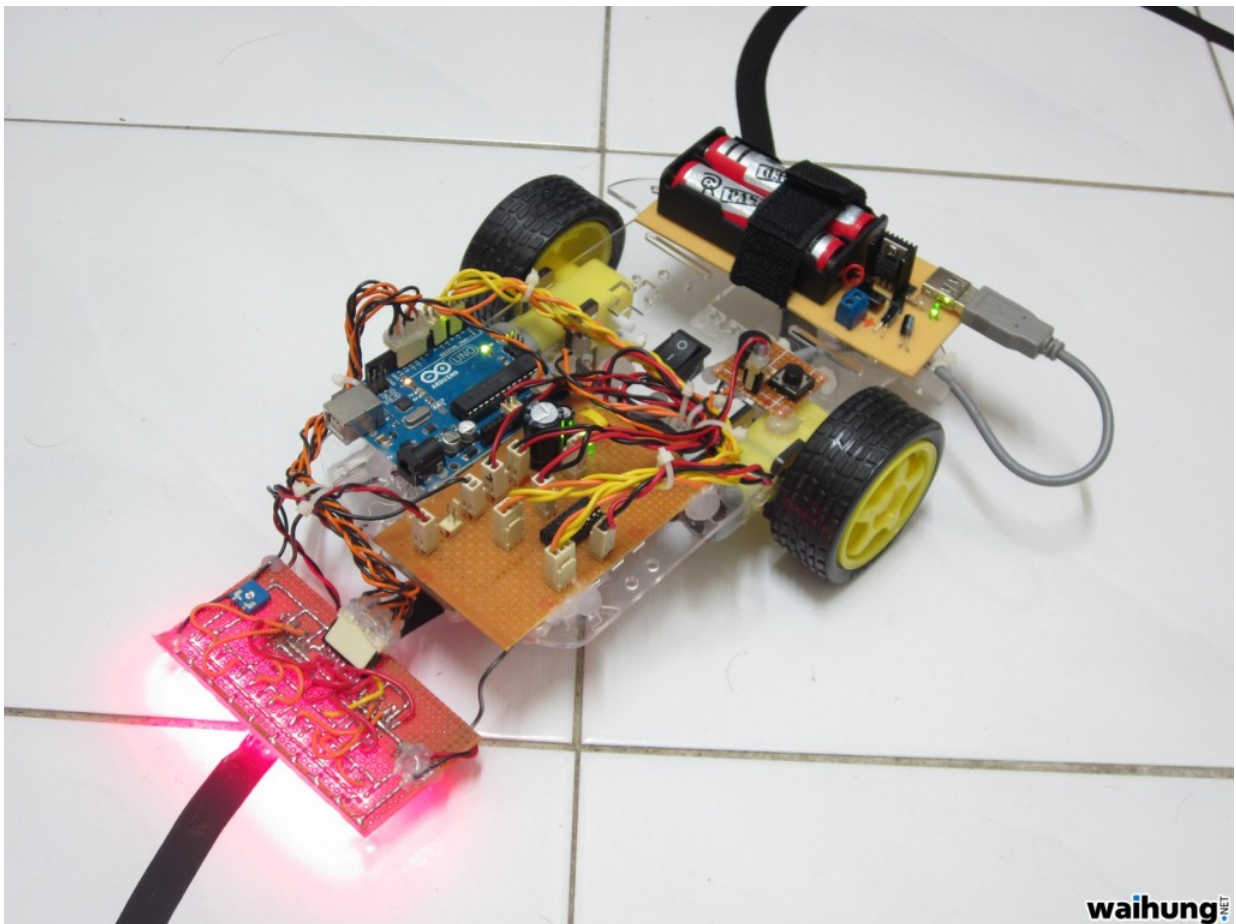


Work of art.

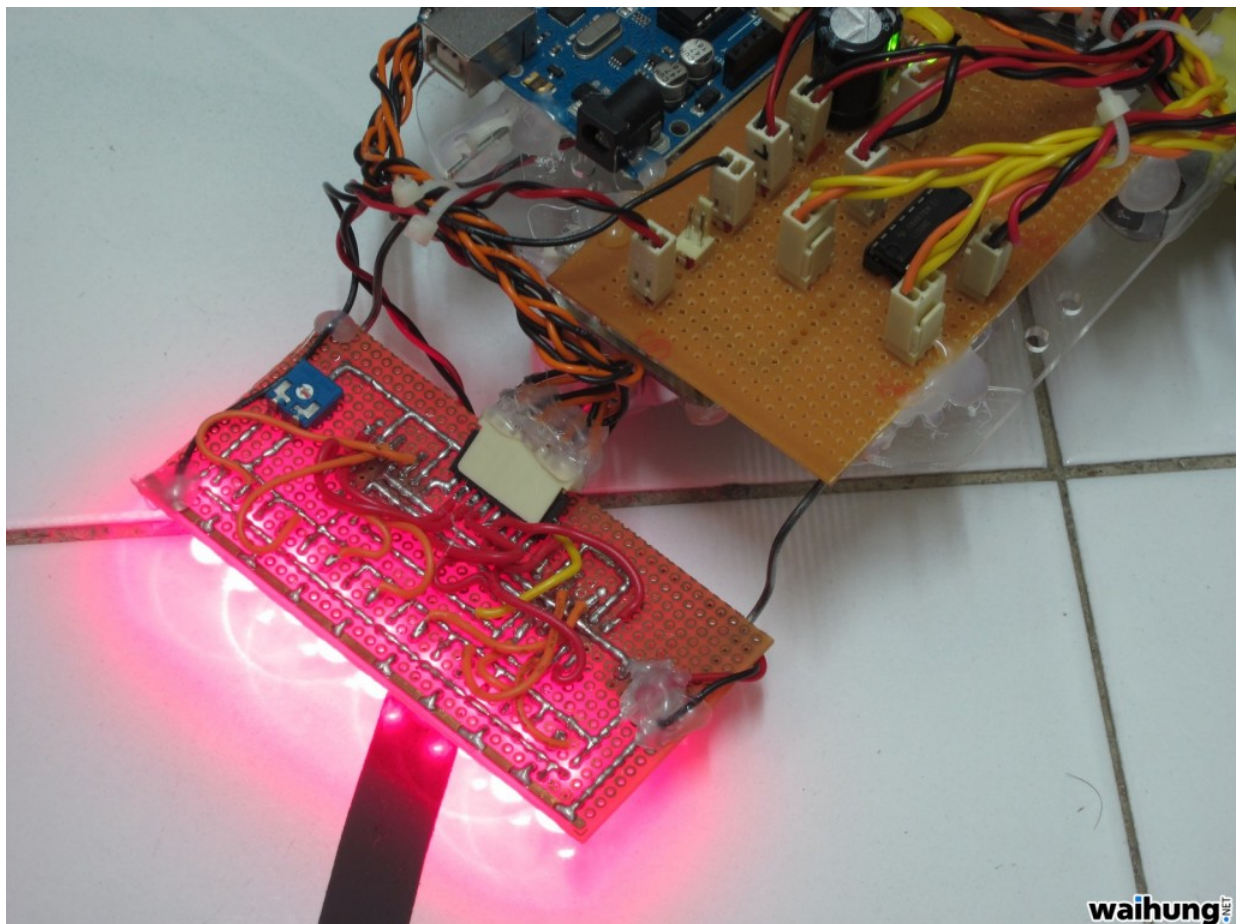
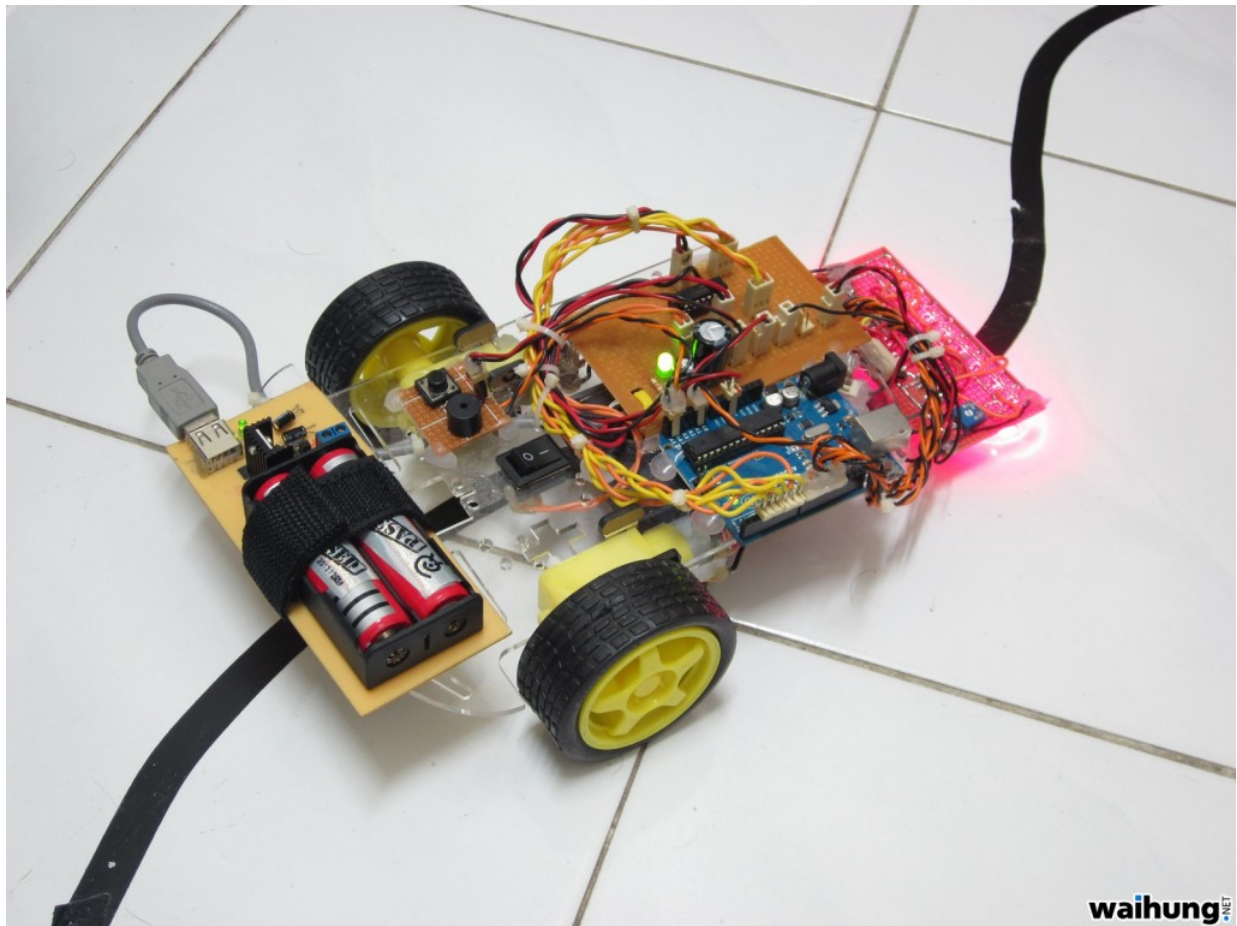


waihung.NET

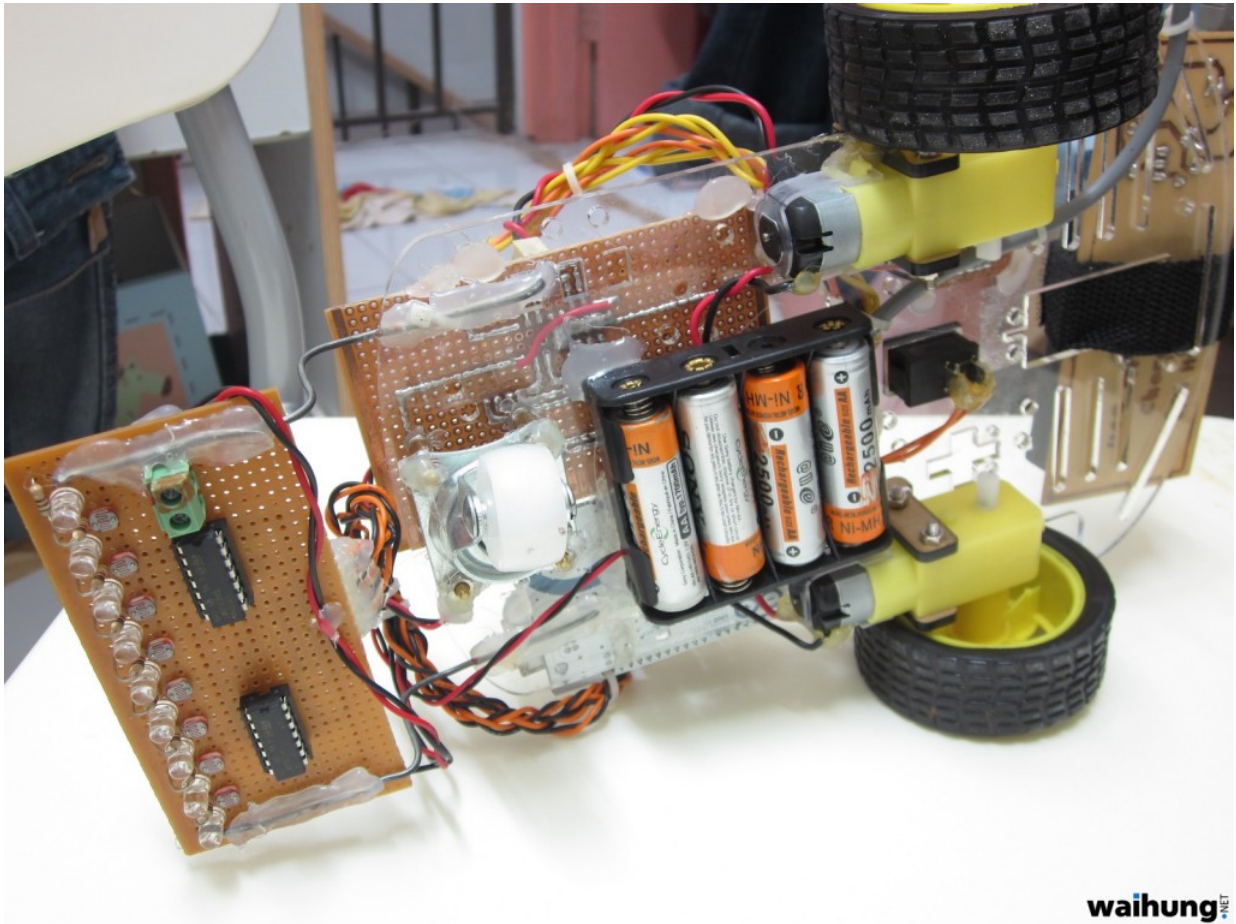
Some glory shots of the robot.



waihung.NET

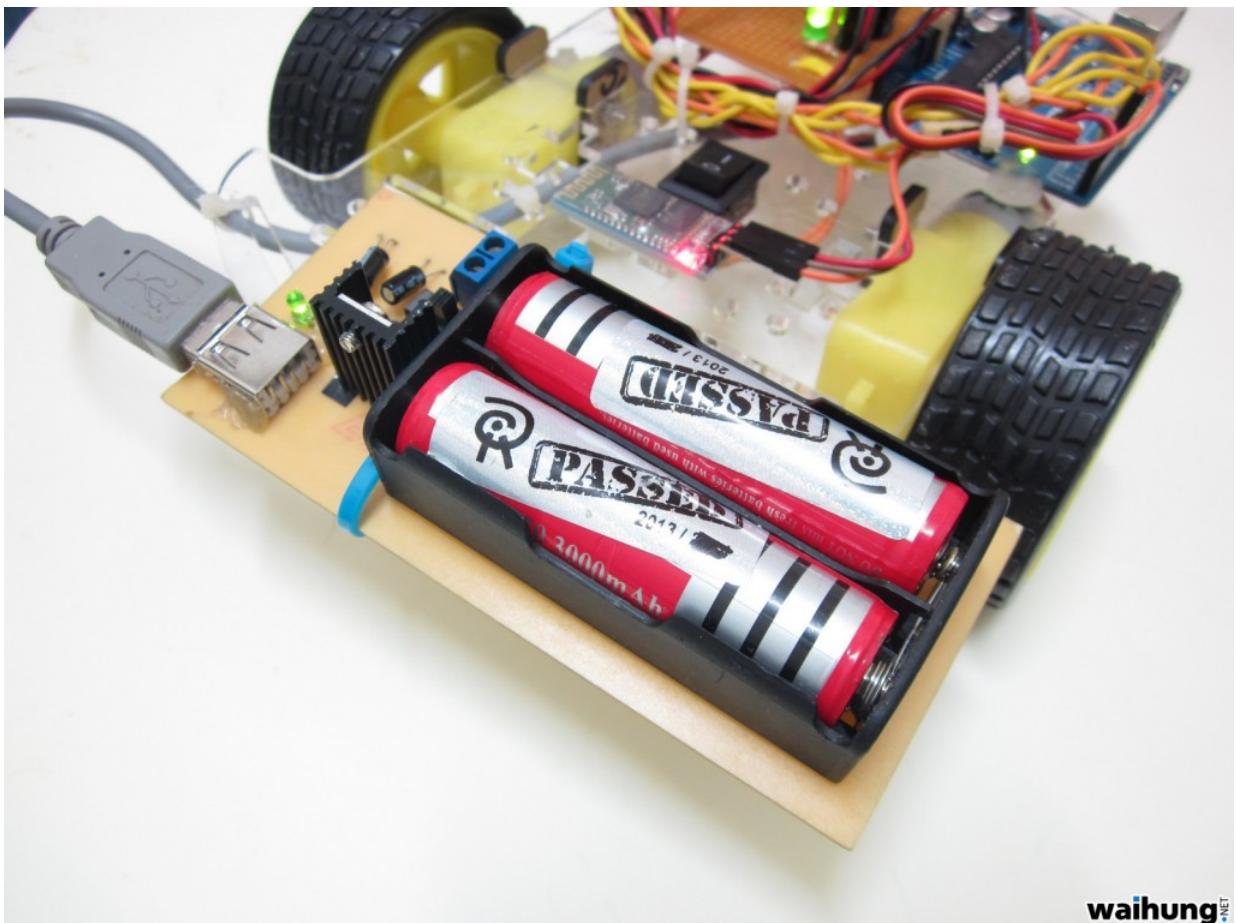


I've added four AA batteries at the underside of the robot to feed the power hungry line sensor.



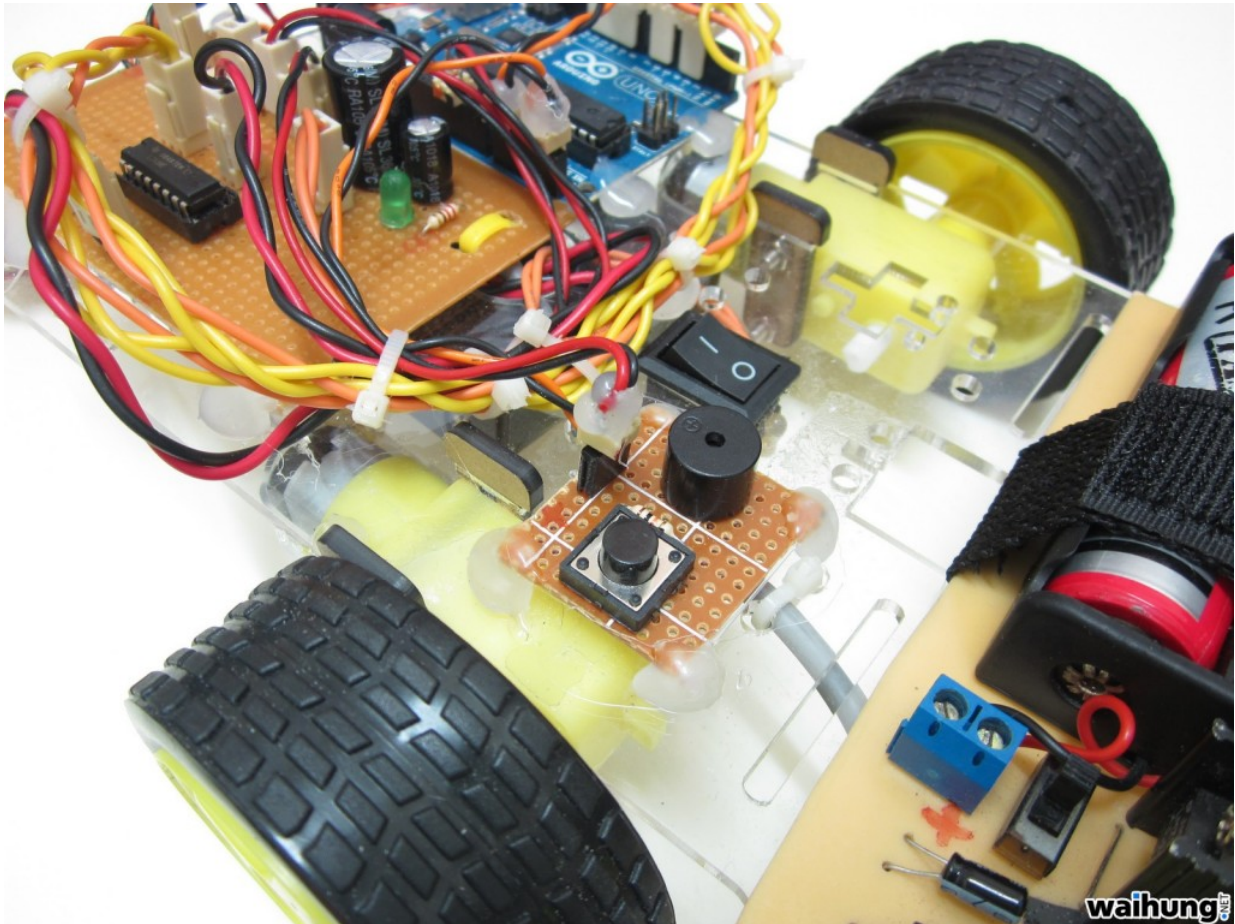
waihung.NET

These lithium-ion 18650 batteries power the electronics and the motor. Anyone recognize the 'Passed' sticker?



waihung.NET

Added a start button and a buzzer.



Now, onto the PID algorithm. PID stands for Proportional, Integral and Derivative.

It is denoted by the following equations. K_p , K_i and K_d are constants which can be tuned to the characteristics of the robot.

Proportional

$$K_p \cdot \text{CTE}$$

Integral

$$K_i \cdot \sum \text{CTE}$$

Derivative

$$K_d \cdot \frac{d}{dt} \text{CTE}$$

PID Equation

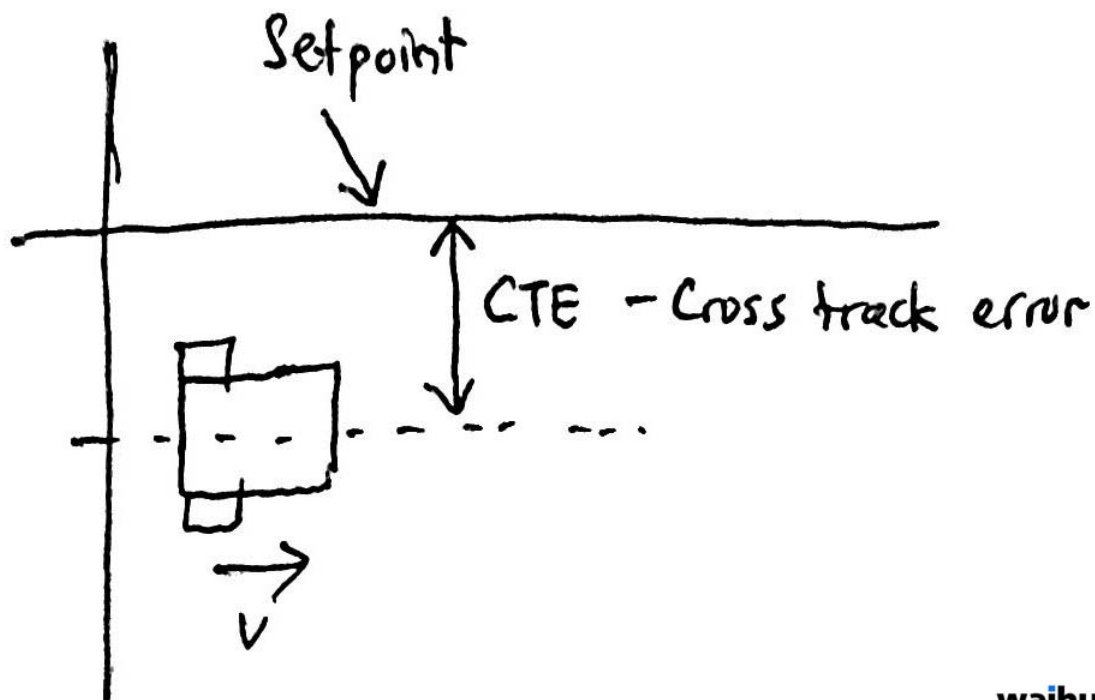
$$\text{steering} = K_p \cdot \text{CTE} + K_i \cdot \sum \text{CTE} + K_d \cdot \frac{d}{dt} \text{CTE}$$

waihung

Confused? I'll try to give examples to understand each of these PID terms better.

First, a bit of introduction.

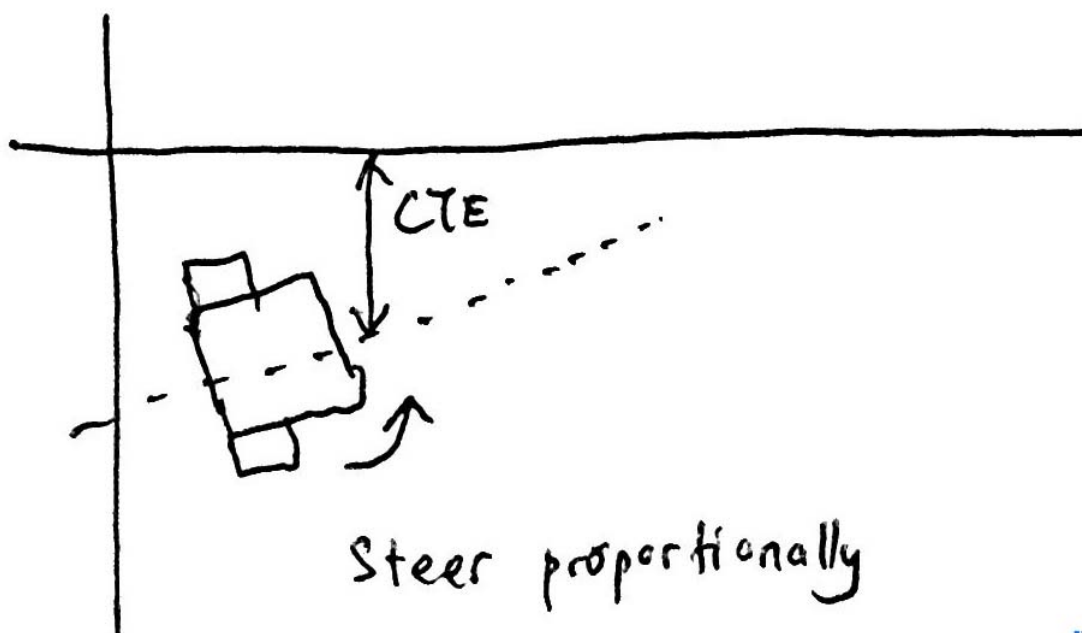
Imagine setpoint is the straight line that your robot should travel but according to the diagram below, it is currently not on the straight line. The position that the robot is in relative to the setpoint is called the Cross Track Error (CTE).



waihung.NET

The P in PID is proportional, meaning the robot will steer in proportion to the Cross Track Error.

The larger the CTE, the harder it steers to the setpoint.

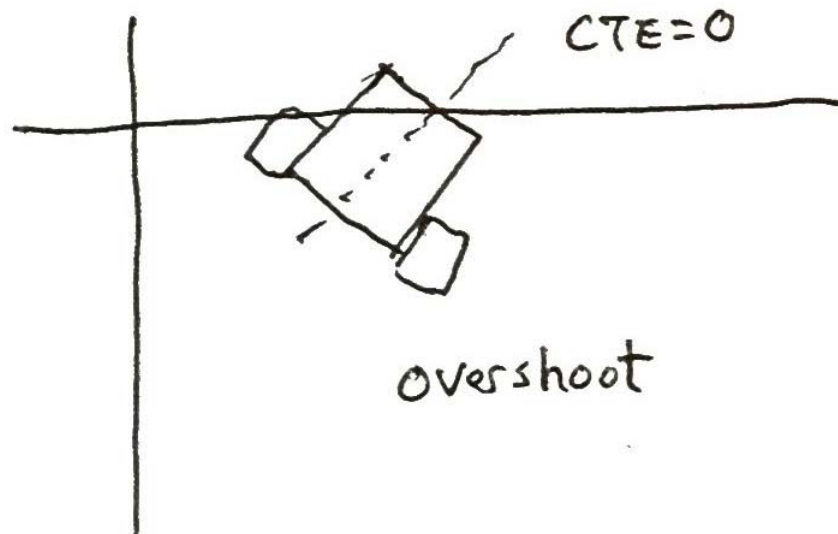


waihung.NET

Therefore, the Cross Track Error will decrease over time as the robot steers to the line.

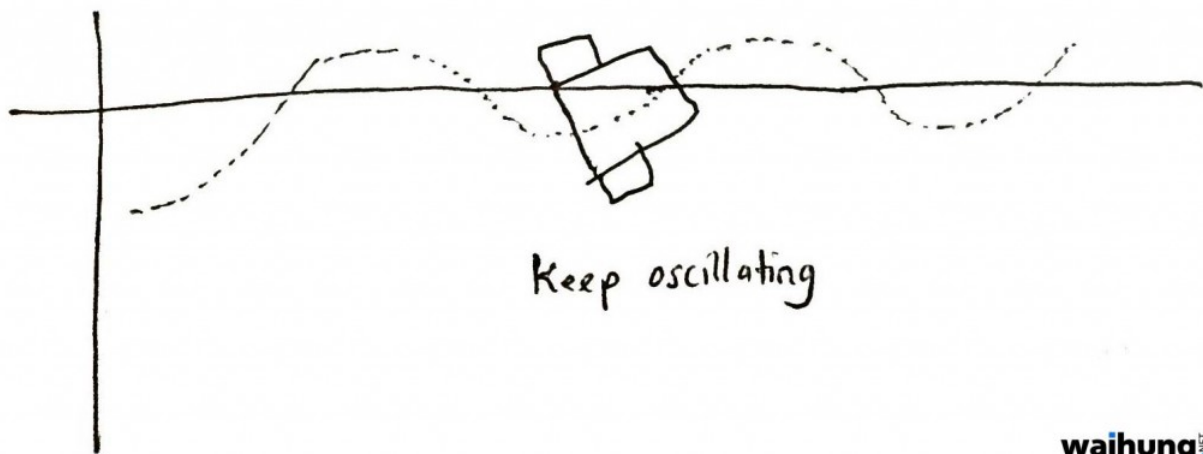
However, picture this diagram below.

The robot will always overshoot because from the diagram below, you can observe that the robot is currently on the line (Cross Track Error is zero) but since it was previously steering at an angle, it is not able to straighten up itself in time when it reaches the line.



waihung.NET

Therefore, a proportional control will always cause the robot to overshoot. It will end up oscillating because it can never get perfectly in line.



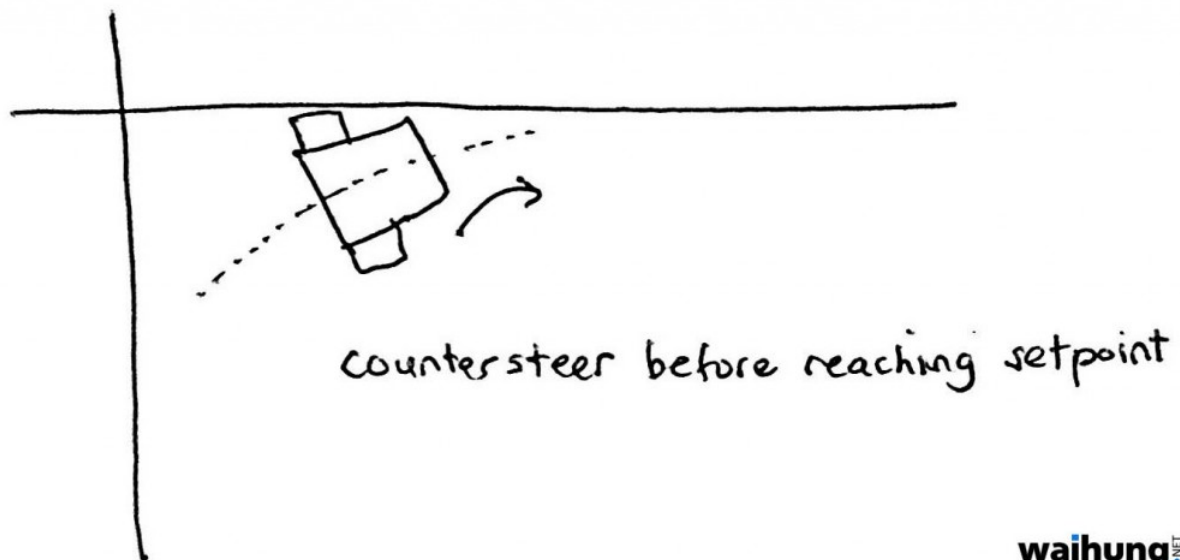
waihung.NET

The way to solve this is to introduce derivative control.

Derivative control is to decrease steering angle based on the difference in Cross Track Error over time.

It basically means that the robot will countersteer when it is about to reach the line.

(If you think about it, when we drive a car or ride a bicycle, we always countersteer before we reach the point where we want to go)

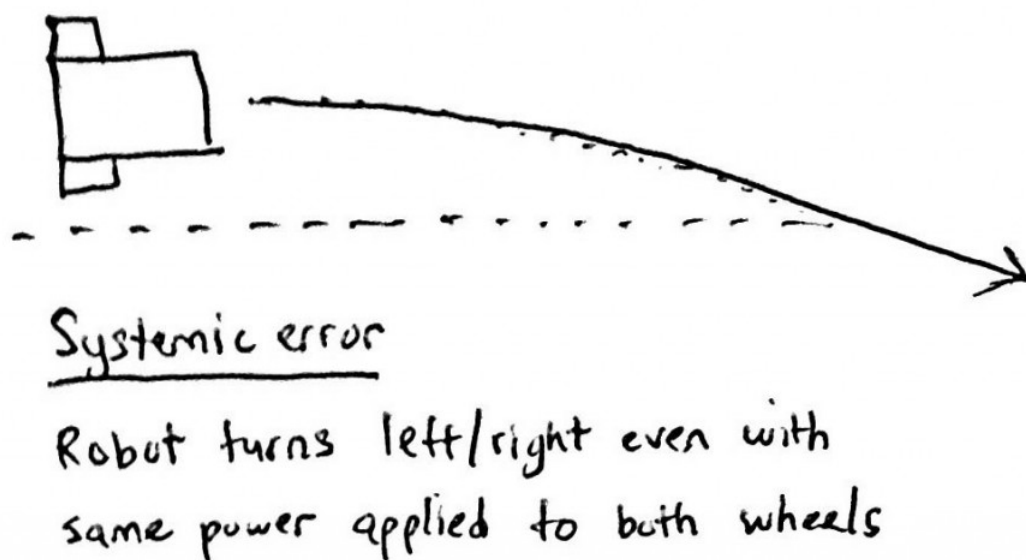


The end result is a smooth curve that ends up straight on the set point after a period of time.

So, we are left with the integral term. What does integral control do?

For those who have played with robots before, you'll notice that robots tend to veer to the left or right even when both wheels get the same power.

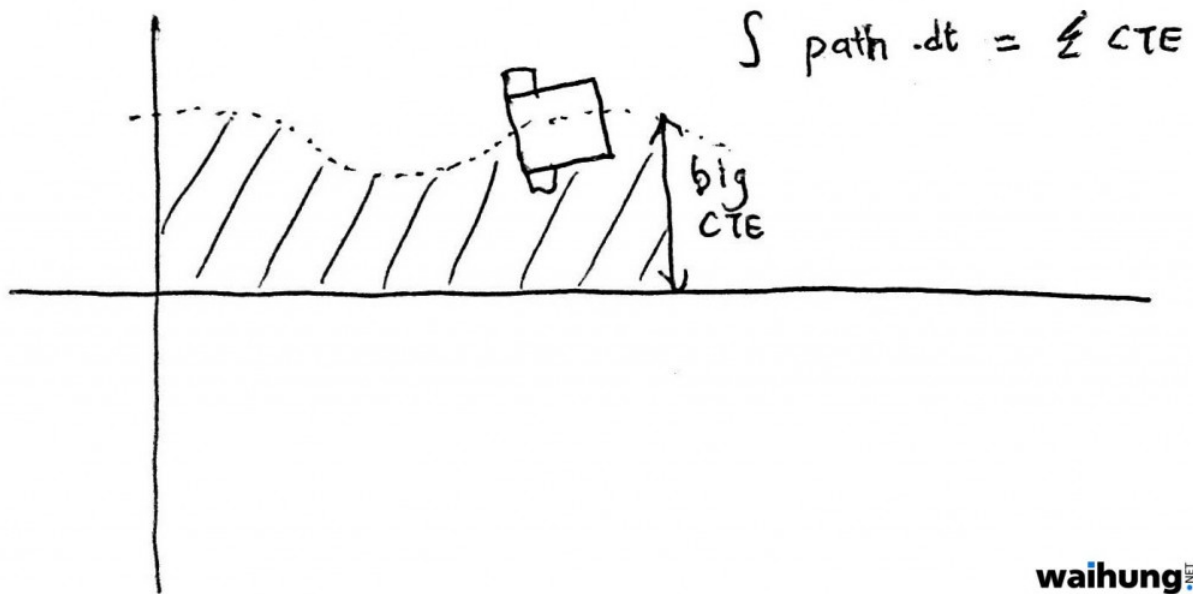
This is called Systemic Error.



Which means it is a mechanical fault where both wheels or motors are not uniform. Every motor and wheel that is manufactured is rarely equal because the manufacturing process always allows a certain amount of tolerance.

For example, if the right motor of the robot moves a little faster than the left motor, the robot will turn left slowly even when the same power is applied to both wheels.

If we just apply Proportional and Derivative control, the following occurs.



The robot will travel straight in parallel to the setpoint, but it is way off its' intended position.

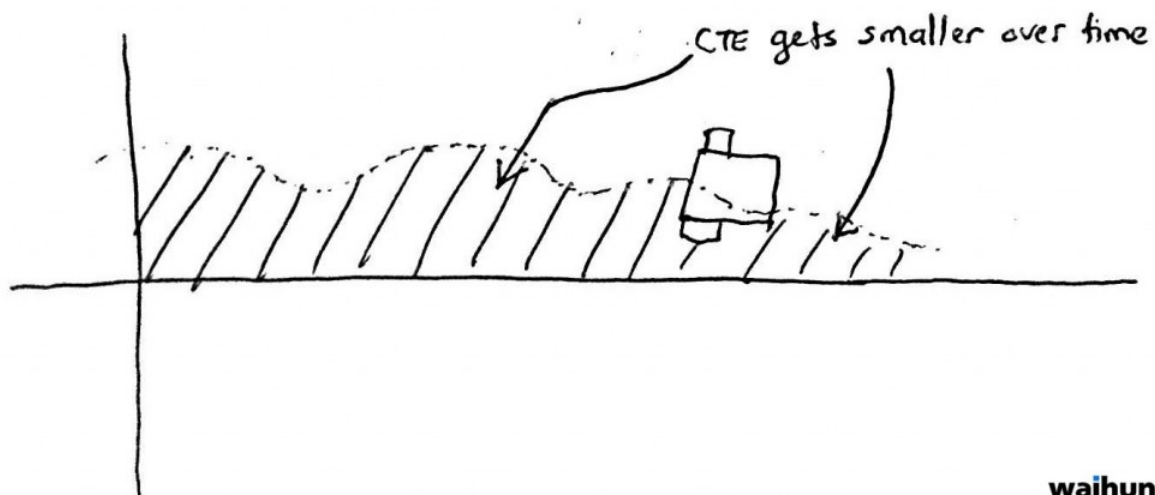
This is where integral control comes in.

Integral control is the integration of the robot path over time, which also means the sum of all the Cross Track Errors over time.

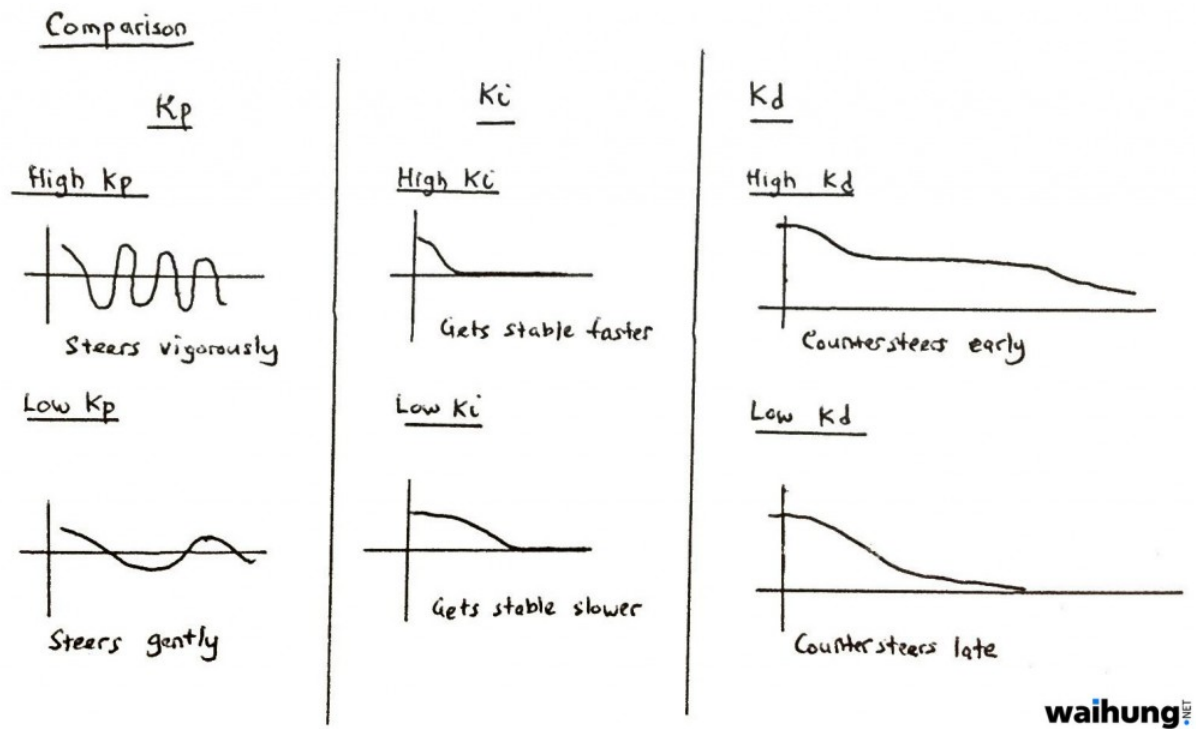
It allows the robot to keep track of its' deviation over the course.

If the sum of CTE is big, it will result in bigger steering angles at the beginning to let the robot turn to the setpoint. Once it has reached the setpoint, the CTE will stop accumulating and hence it will stay at the setpoint.

In conclusion, integral control creates an offset to eliminate or reduce any systemic errors.

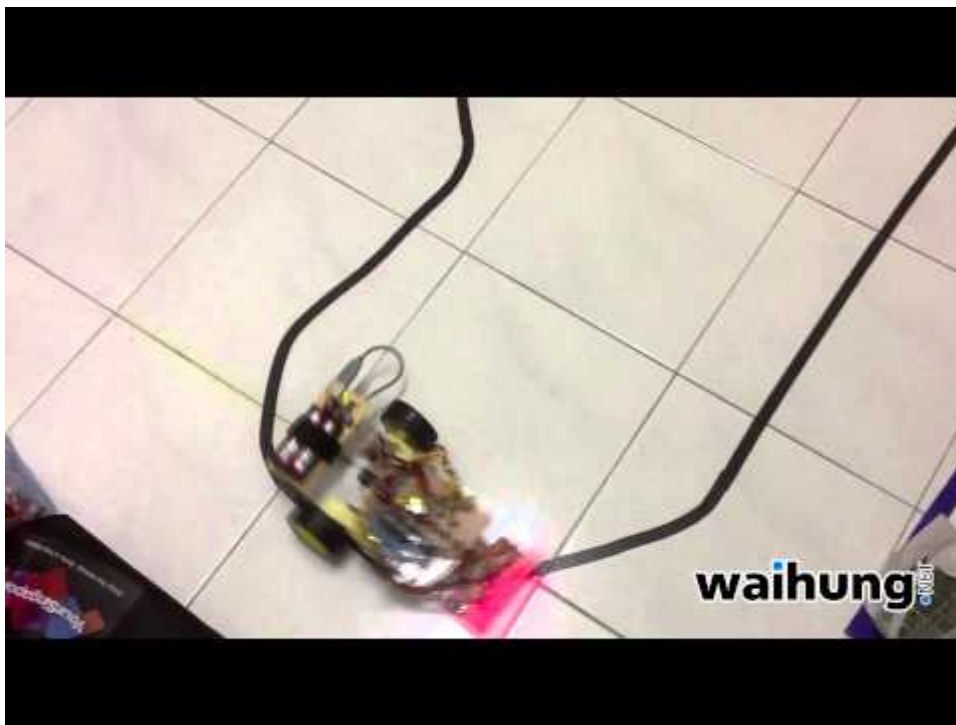


Below is an overview of the PID constants and what it does.



Hopefully all the information above will help you understand PID better.

This is a video of my robot going around a track.



Watch Video At: <https://youtu.be/Oa1SCP6jUD4>



This is my code written in Arduino IDE.



[Download Code](#)

Thanks for reading.